



18-649 Guest Lecture: **Machine Learning**

Tianshu Huang — CMU WiseLab | LIONS

- (1) When should I use ML / DL?
- (2) How I develop and deploy ML to the edge?



This lecture brought to you by:

Learning for Systems

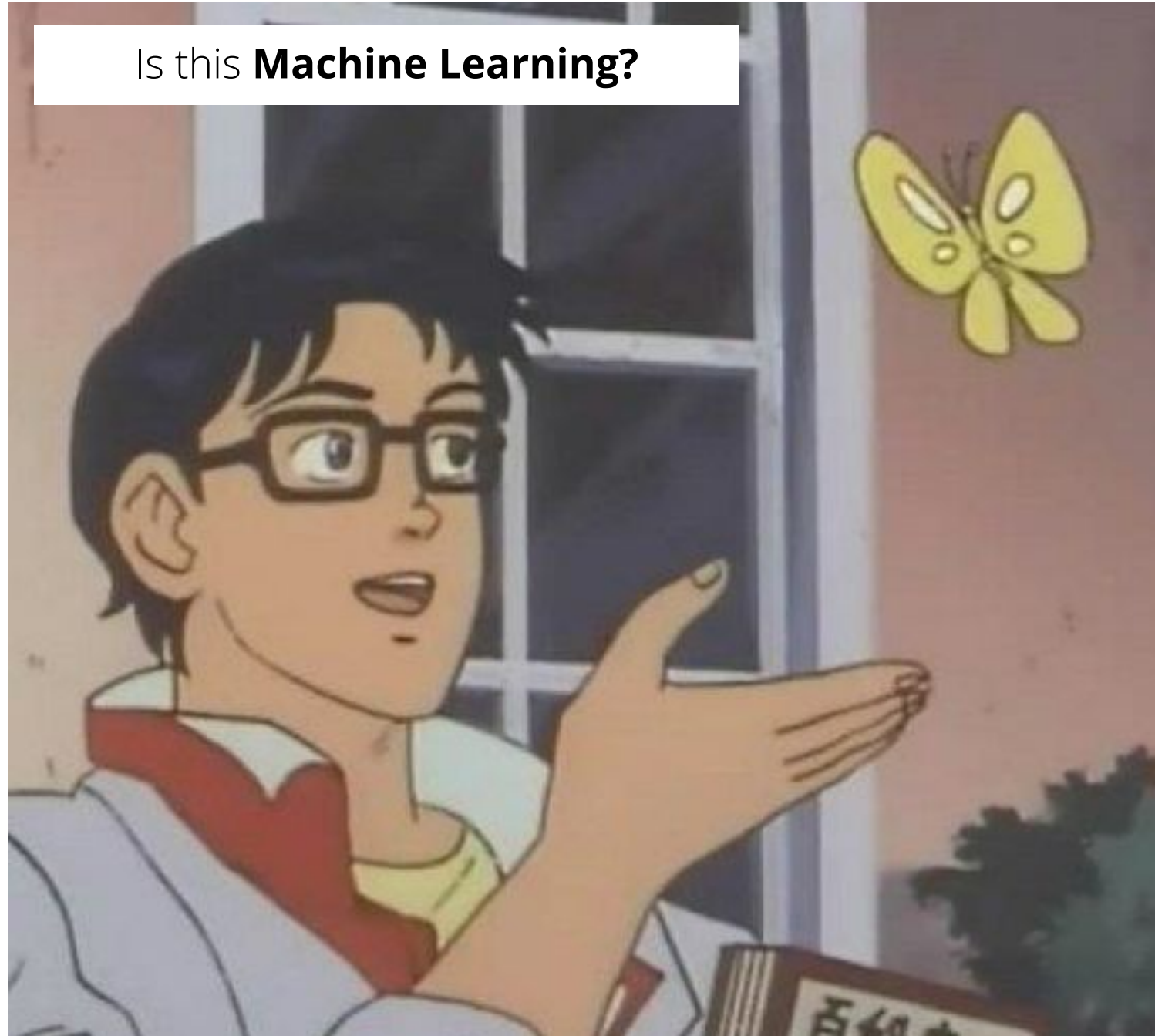
Have some background in Machine Learning and/or systems, and interested in **using Machine Learning to understand and improve systems?**

- Understanding program resource usage
- Edge orchestration
- Handling network dropout
- And more!

Contact: tianshu2@andrew.cmu.edu

Machine Learning

Is this **Machine Learning**?

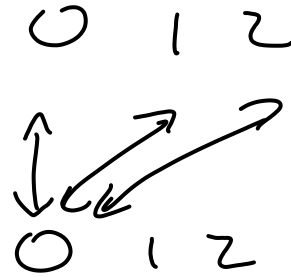


Machine Learning

Example: Digit Classification

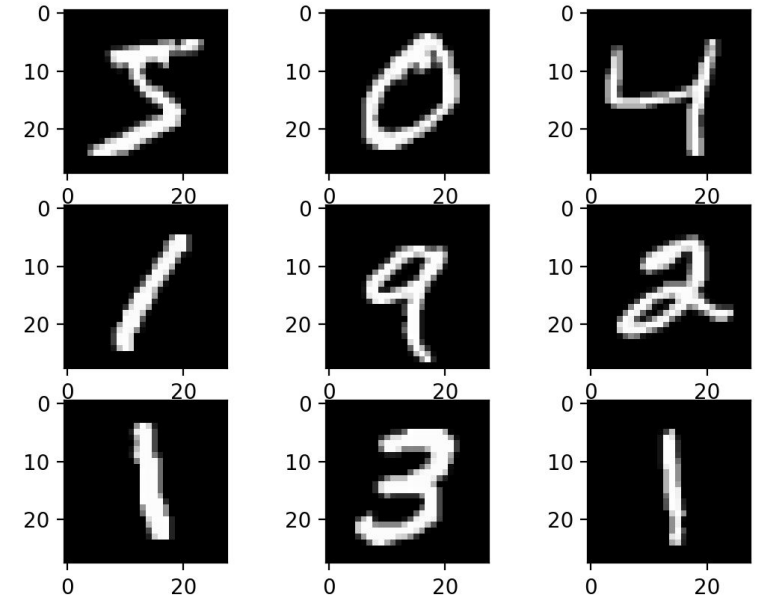
No Data:

- Color distribution
- pixel neighborhoods
- Hough transform



Some Data

- Compare images
- feature extraction
+ examples



Machine Learning

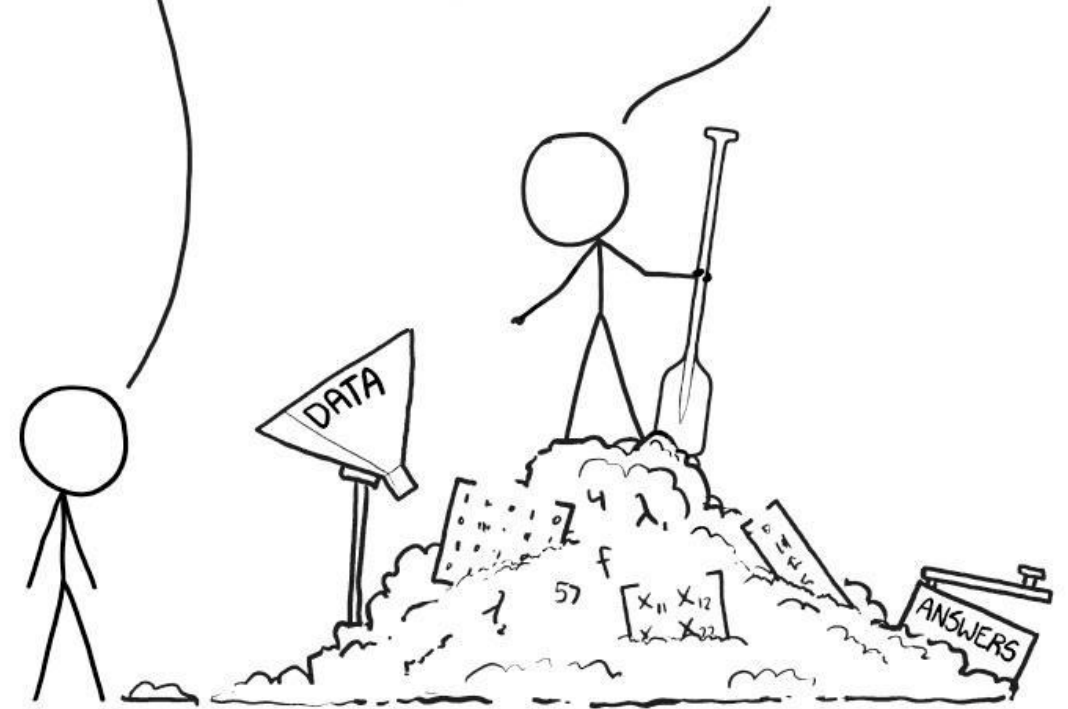
- NN
- Conv NN
- Naive Bayes
- Linear Regression
- XGBoost
- Random Forest
- Decision Trees
- SVM
- Transformers
- Perceptrons
- PCA

THIS IS YOUR MACHINE LEARNING SYSTEM?

YUP! YOU POUR THE DATA INTO THIS BIG PILE OF LINEAR ALGEBRA, THEN COLLECT THE ANSWERS ON THE OTHER SIDE.

WHAT IF THE ANSWERS ARE WRONG?

JUST STIR THE PILE UNTIL THEY START LOOKING RIGHT.



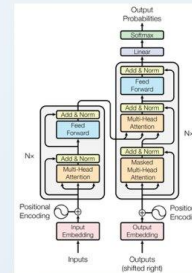
Deep Learning

STOP DOING DEEP LEARNING

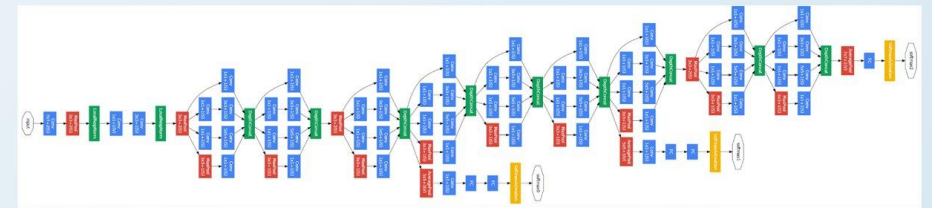
- PERCEPTRONS WERE ONLY EVER MEANT TO BE FULLY CONNECTED
- THOUSANDS OF PAPERS yet NO REAL-WORLD USE FOUND for going deeper than ONE LAYER
- Wanted to add more nonlinearity anyway for a laugh? We had a tool for that: It was called “KERNEL METHODS”
- “Yes please give me a network that can PAY ATTENTION TO ITSELF. Please give me PRETRAINED WEIGHTS for my YOLO-9000” - Statements dreamed up by the utterly Deranged

LOOK at what “Research Scientists” have been demanding your Respect for all this time, with the statistical methods & optimization algorithms we built for them

(This is REAL “Deep Learning”, done by REAL “ML Engineers”):



?????

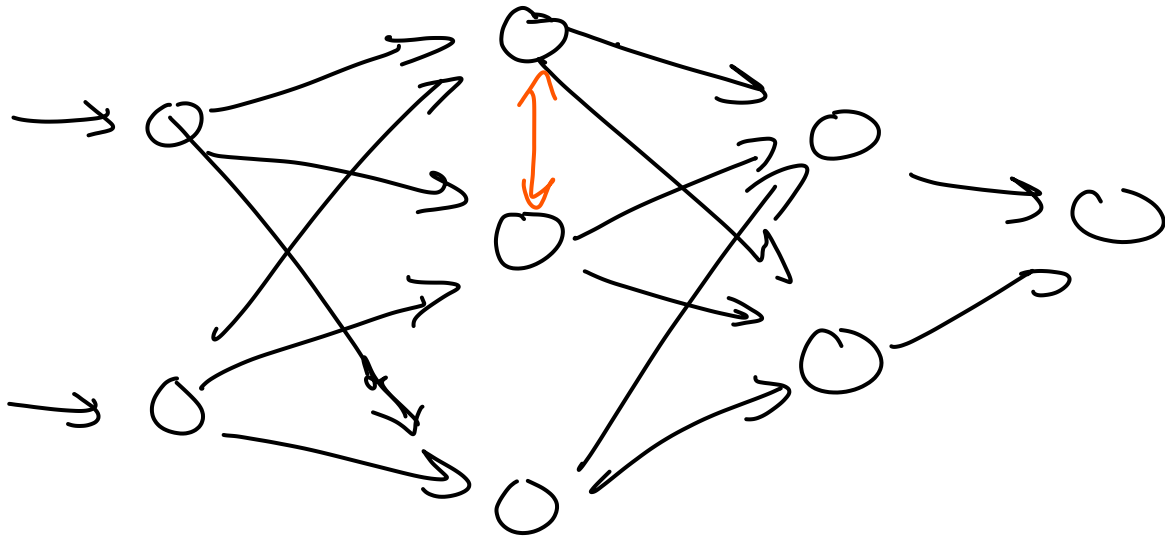


????????????????????

“Hello I would like to learn **1.6 TRILLION** parameters please”

They have played us for absolute fools

Deep Learning



"Dense"

"MLP"

"Fully Connected"

The Mysteries of Deep Learning

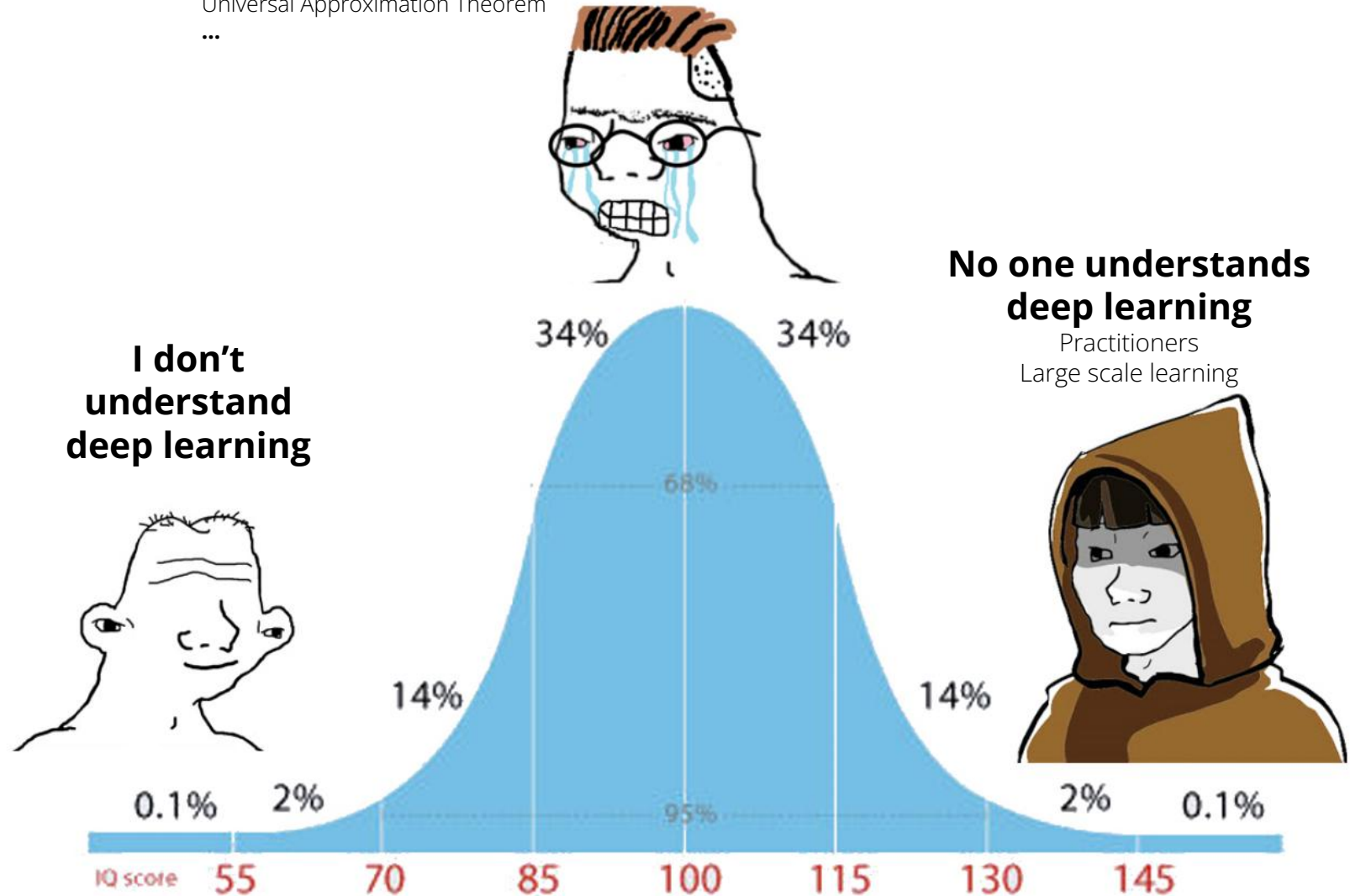
NOoO!! You have to understand Deep learning!!!

Merging Models modulo Permutation Symmetries (Ainsworth et al., 2022)

Neural Networks are Decision Trees (Aytekin, 2022)

Universal Approximation Theorem

...



The Natural Image Manifold

The **natural image manifold** is a surface embedded in a **high dimensional space** that is:

- (1) **low-dimensional**,
- (2) **highly nonlinear**, and
- (3) **locally smooth**.

- high dim: $100 \times 100 \text{ px}$, grayscale
 $10,000$

- low-dim subspace!

$\text{Unif}[\text{image}] \approx \text{noise}$

- nonlinear

$\frac{\text{img}_1 + \text{img}_2}{2} \notin \text{image}$

- locally smooth

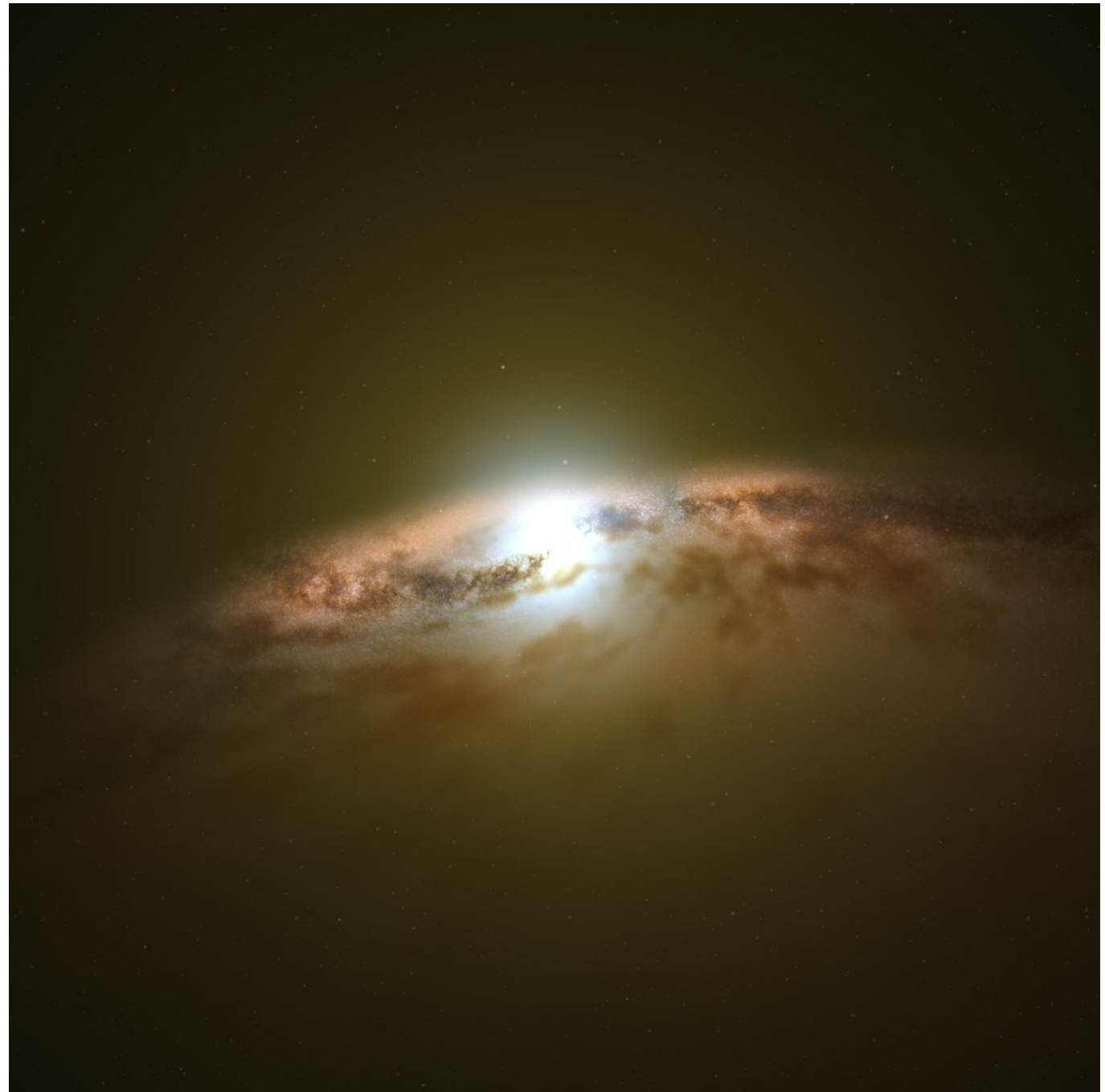
$\text{image} + \varepsilon \approx \text{image}$

The Natural Image Manifold

The **natural image manifold** is a surface embedded in a **high dimensional space** that is:

- (1) **low-dimensional**,
- (2) **highly nonlinear**, and
- (3) **locally smooth**.

Analogy by the late Thomas S. Huang via Atlas Wang



The Natural Image Manifold

The **natural image manifold** is a surface embedded in a **high dimensional space** that is:

- (1) **low-dimensional**,
- (2) **highly nonlinear**, and
- (3) **locally smooth**.

Good:

Computer Vision

Audio Processing

Graphics

Natural Language

↳ Text understanding

↳ source code

Bad:

Sales

Anything with a
simple analytical
soln.

Tabular Data

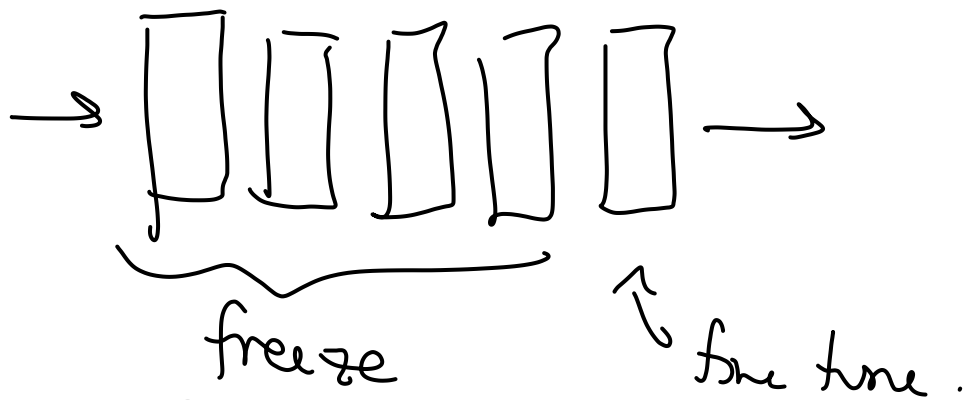
Practical Deep Learning



See **Adversarial Robustness**.
Image from "Robust Physical-World
Attacks on Deep Learning Models,"
Eykholt et al, 2018.

Pre-Trained Models & Transfer Learning

Google
(ResNet, MobileNet,
YOLO)



13



Practical Deep Learning

- Train from scratch?
- Transfer learn?
- Fully pre-trained model?

Eleuther AI



Deep Learning on the Edge



Please inspect your power cables if you own a RTX 4090, even if you don't use it for deep learning.

Deep Learning on the Edge

- Trained neural network with a given architecture
- Weights are 32-bit float
- Given power budget

$f32 \rightarrow i8$ (Quantization)

Pruning

Distillation

Reduction (space, time)

offloading

More efficient

Simplest is Best

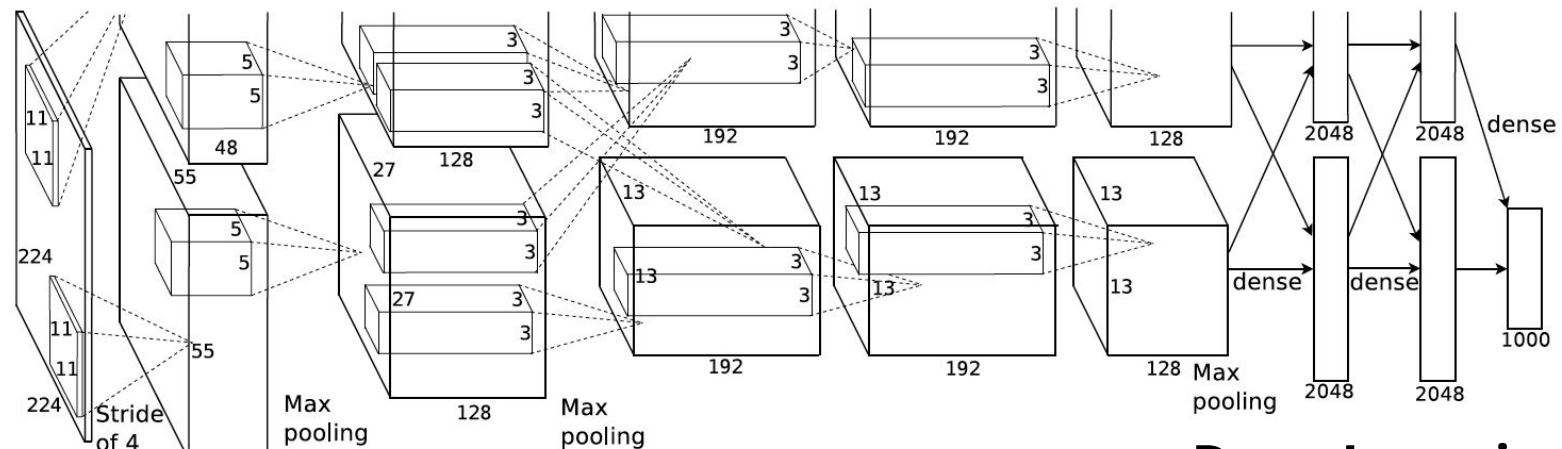
- Cloud Offloading
- Reduce input resolution
- Don't use deep learning



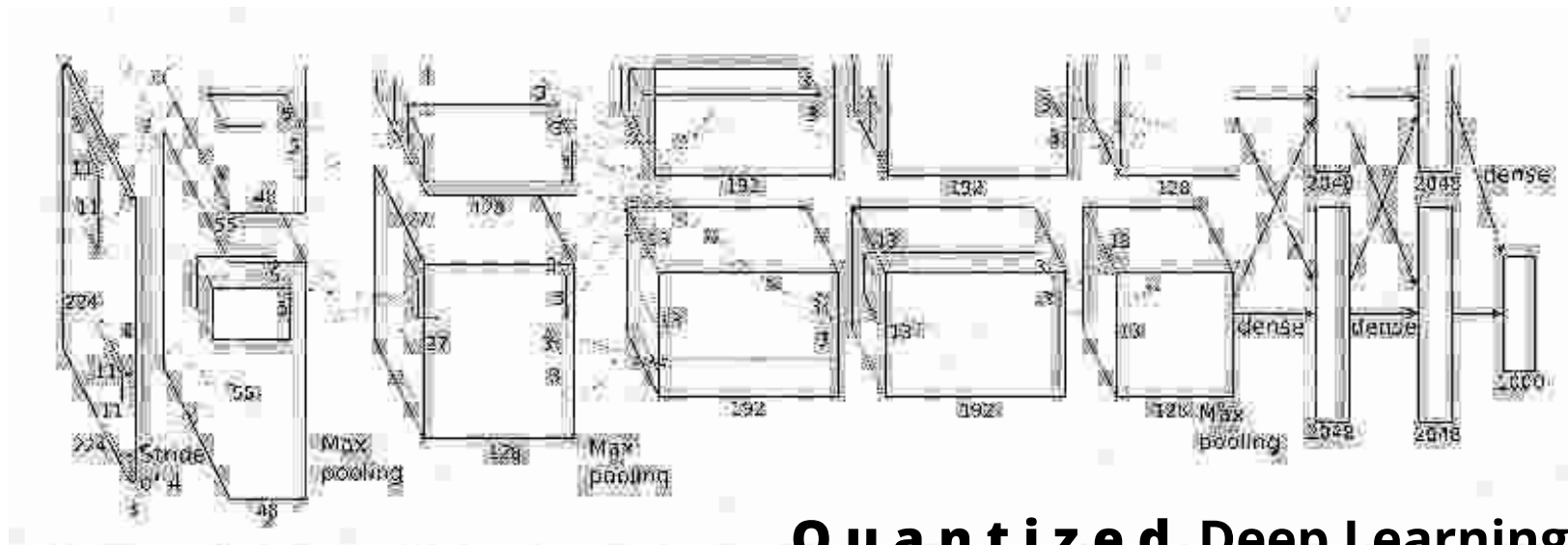
Quantization

- float32 $\rightarrow$ float16
- Even better: int8
- Binary is even possible

"ImageNet Classification with Deep Convolutional Neural Networks," Krizhevsky et al., 2012 (AlexNet - First "true" ConvNet)



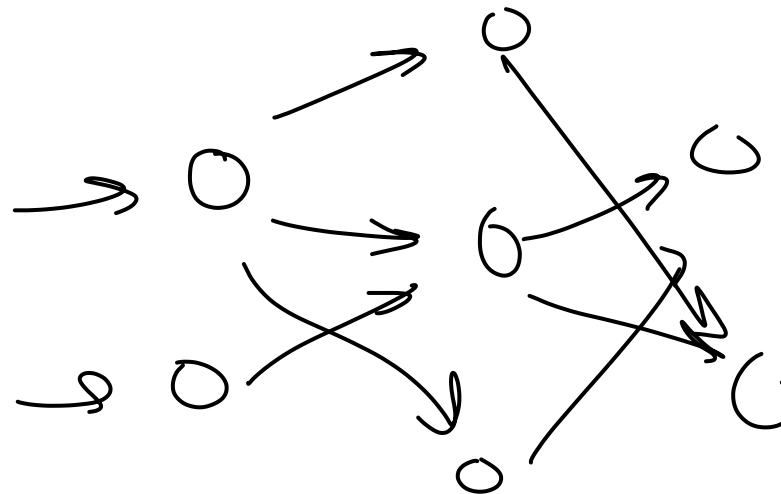
Deep Learning



Quantized Deep Learning

Pruning

- Pruning: Optimal Brain Damage (LeCun et al., 1989)
- Structured Sparsity
- Lottery Ticket Hypothesis



Pruning

- Pruning
- Structured Sparsity
- Lottery Ticket Hypothesis



**Baseline
Model**



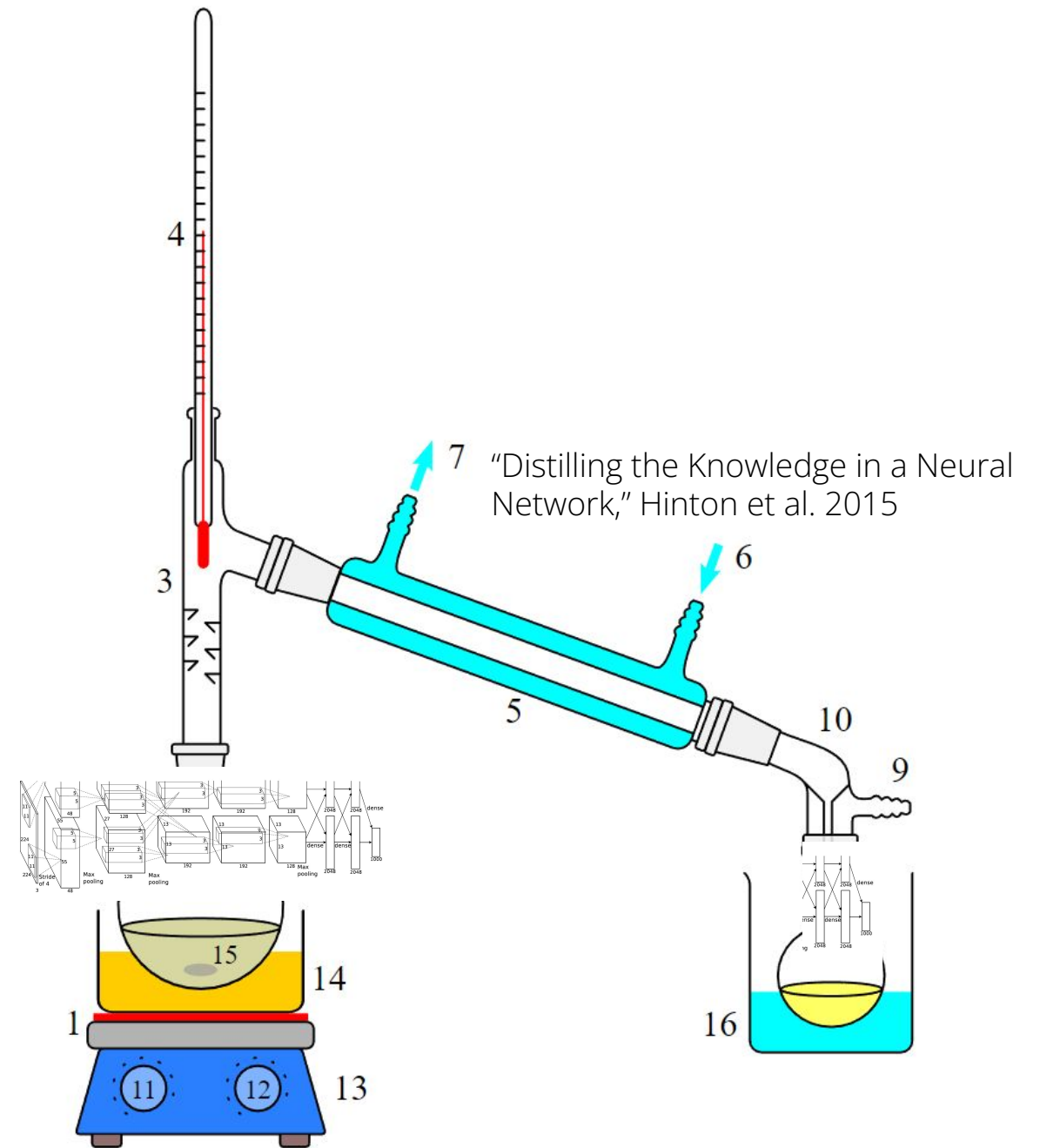
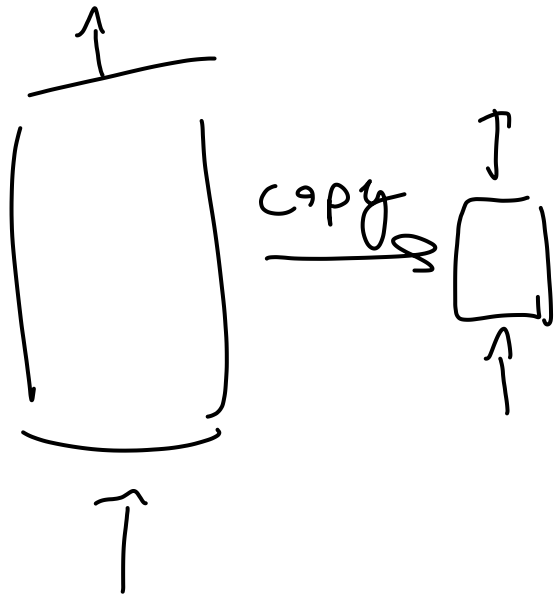
**Quantization
Optimized
Execution**



**Optimal Brain
Damage**

LeCun et al., 1989

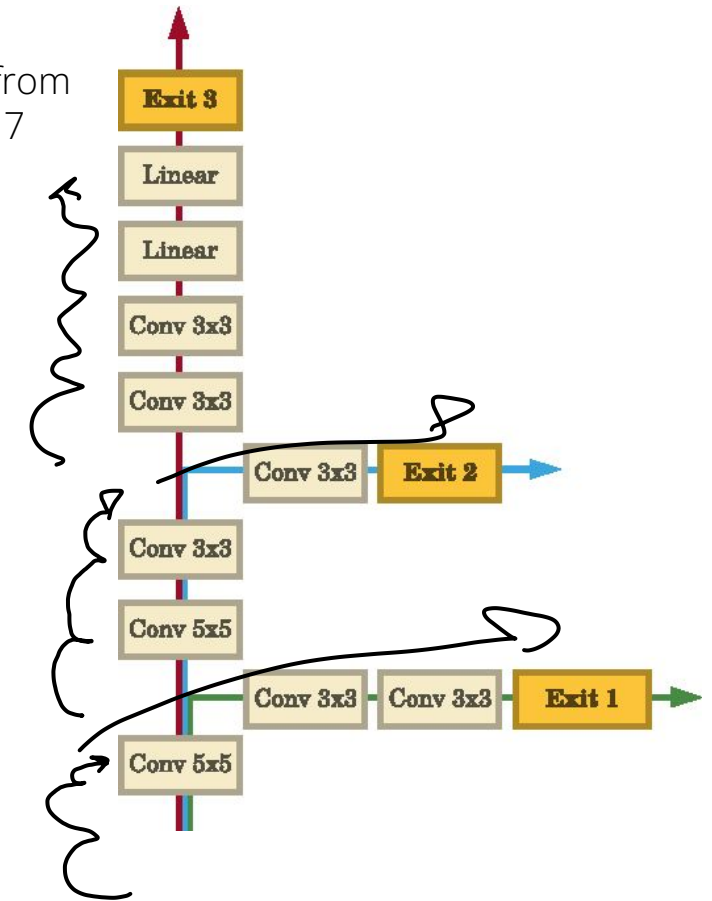
Distillation



Adaptive Inference

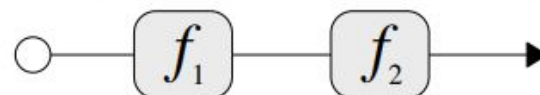
- Different numbers of layers
- Different input resolution
- ...

"BranchyNet: Fast inference via early exiting from deep neural networks," Teerapittayanon, 2017

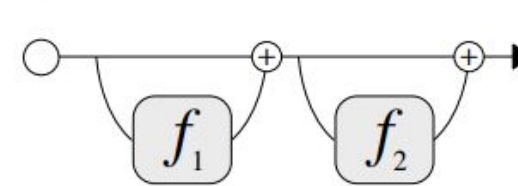


"Convolutional Networks with Adaptive Inference Graphs," Veit & Belongie, 2018

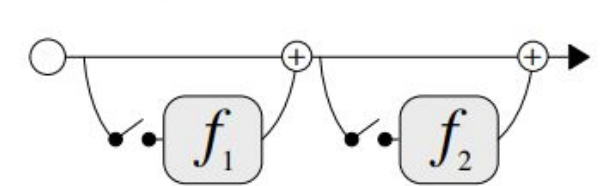
Traditional feed-forward ConvNet:



ResNet:



ConvNet-AIG:





TL;DR

1. Don't use machine learning
2. Don't use deep learning
3. Don't use deep learning on the edge (do it in the cloud)
4. Don't train deep learning models (use a pre-trained model optimized for edge deployment)
5. Don't train deep learning from scratch (use transfer learning)
6. Give up (or take a machine learning class)



Additional Topics

- Distributed / federated training
- Edge Training
- GPU vs CPU vs TPU
- Deep Learning Frameworks
- Edge Deployment Frameworks (TF Lite, Pytorch Mobile)
- ML Accelerator Design
- ML Compilers
- Efficient Architectures
- Anything ML Related

